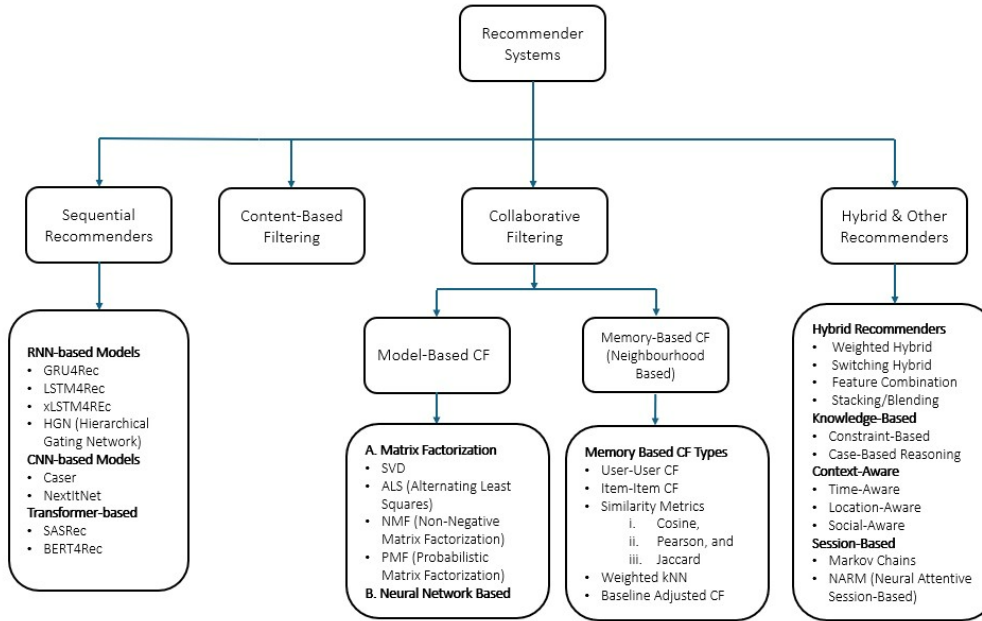


Recommender Systems – High Level Overview



General Classification of Recommender Systems:

- **Sequential Recommendation (SR):-** SR focuses on next-item prediction by modeling the temporal ordering of user interactions. This is the **primary research focus in this thesis work**.
- **General Recommendation (GR):-** User–item interaction data, implicit feedback, such as clicks, add-to-cart events, purchases, time spent, or interaction frequency.
- **Content-Aware Recommendation:-** Click-through rate (CTR) prediction tasks, using explicit feedback and binary classification. As feature-based, go beyond raw user–item interactions users, items, or context.
- **Knowledge-Based Recommendation:-** Utilizes external knowledge graphs to add semantic or structural context beyond interactions.

Problem Statement

- Despite the advancements brought by deep learning in improving recommendation accuracy, **several critical challenges** persist in modeling user–item sequences at scale.
- These challenges include **limited scalability, popularity bias, the cold-start problem, data sparsity, transfer learning issues**, and the need for robust long-term dependency tracking across user interactions.
- Recurrent architectures such as LSTM, though effective in capturing temporal dependencies, **suffer from limited scalability and are inherently sequential, making them less efficient** for large-scale recommendation.
- In contrast, Transformer-based models like BERT4Rec and SAS4Rec offer greater parallelism and improved performance through self-attention mechanisms but often come with **high computational costs and memory requirements**.
- We investigate whether the recently **proposed xLSTM architecture**, which **integrates bidirectional memory, attention mechanisms, and enhanced gating structures**, can serve as a ground—achieving scalability, computational efficiency, and high accuracy in sequential recommendation tasks, while addressing the challenges discussed above.

Research Questions and Contributions of This Work

This report aims to answer the following **four research questions**:

- **RQ1:** How does xLSTM's performance scale with dataset size compared to established architectures like BERT4Rec and SAS4Rec ?
- **RQ2:** How do sequence length and embedding size influence model performance across different item-popularity levels, and do larger sequences or embeddings improve a model's ability to make accurate long-tail recommendations ?
- **RQ3:** What trade-offs exist between recommendation accuracy and computational cost as sequence length and model complexity increase?
- **RQ4:** Embedding Saturation and Utilization: How do different model architectures make effective use of their embedding representations, and does embedding dimensionality lead to better spatial distribution, representation diversity, or improved predictive performance ?

Following **key contributions** to the field of sequential recommender systems:

- **Implementation and Evaluation of xLSTM:** We present a **novel adaptation and empirical evaluation** of the existing xLSTM architecture for sequential recommendation tasks, using benchmark datasets such as MovieLens for reproducibility and comparative analysis.
- **Comprehensive Model Benchmarking:** We conduct a rigorous comparison between xLSTM and several baseline models, including Transformer-based (e.g., BERT4Rec, SAS4Rec) architectures. The evaluation spans both quantitative (e.g., HR@K, NDCG@K) and qualitative performance metrics.
- **Analysis of Computational Efficiency and Scaling Behavior:** We examine model stability, training time, inference characteristics, and resource consumption to understand accuracy–efficiency trade-offs (RQ3).
- **Embedding Utilization and Representation Analysis:** In-depth study of embedding saturation, dimensionality behavior, and latent-space across models, providing insights into how different architectures use embedding capacity (RQ4).

2. Existing Model Architectures For Sequential Recommenders

- Recurrent Neural Networks (RNN)
- Long Short-Term Memory (LSTM)
- Gated Recurrent Unit for Recommendation (GRU4Rec)
- Self-Attentive Sequential Recommendation (SAS4Rec)
- Bidirectional Encoder Representations for Recommendation (BERT4Rec)
- **Proposed Method:** xLSTM for Sequential Recommendation

Feature	LSTM	xLSTM	BERT4Rec	SAS4Rec
Attention like Context	No	Yes	Yes	Yes
Bi-directional	No	Yes	Yes	No
Training Stability	Medium	High	Medium	High
Inference Speed	Fast	Moderate	Slow	Moderate

Self-Attentive Sequential Recommendation (SAS4Rec)

- SAS4Rec **uses self-attention mechanisms to effectively capture long-term user behavior patterns** SR tasks, making it central for evaluating competitiveness in RQ1 and RQ3.
- **Left-to-right manner, Unidirectional.**
- The architecture employs **Transformer-style self-attention** blocks and positional embeddings to preserve item order, along with residual connections and layer normalization to stabilize training.
- Built on Transformer blocks with **positional embeddings, residual connections, and layer normalization.**
- SAS4Rec represents a significant advancement in efficient, interpretable, and accurate sequential recommendation.

Bidirectional Encoder Representations for Recommendation (BERT4Rec)

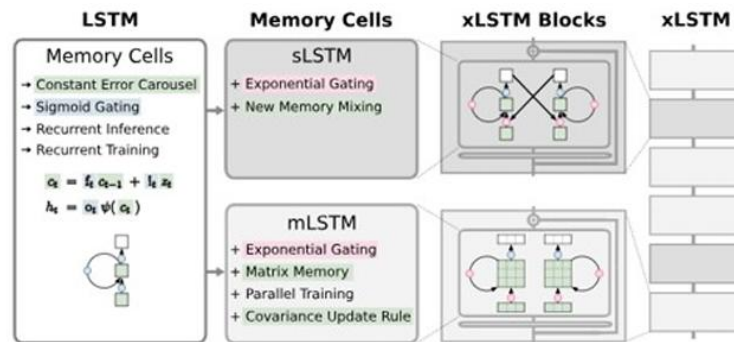
- **Strongest Transformer baseline**, making it essential for benchmarking xLSTM (RQ1, RQ2) and for comparing embedding behavior (RQ3, RQ4).
- **Bidirectional** learns from both past and future items, improving its ability to model complex sequence semantics.
- The model architecture stacks multiple Transformer layers with **multi-head self-attention, residual connections, and layer normalization.**
- **Enabling deep bidirectional context modeling**, can trivialize training, as each item could indirectly access the target item.
- BERT4Rec trained using a masked-item (Cloze) objective, which predicts randomly masked items in sequence based on both their left and right contexts.

4. Proposed Method: xLSTM for Sequential Recommendation

xLSTM integrates two powerful variants:

- **sLSTM (scalar LSTM)**: Enhances the classical memory cell with exponential gates and improved normalization to stabilize long-term temporal modeling. (with scalar memory, scalar update, and memory mixing),
- **mLSTM (matrix LSTM)**: Extends recurrence into a matrix-valued memory, enabling the model to store and retrieve more structured sequence information with greater expressive capacity. (with a matrix memory and a covariance update rule, parallelable)

xLSTM provides the architectural foundation for evaluating performance scaling (RQ1) and sequence-length sensitivity (RQ2). Its hybrid memory mechanisms also offer a unique opportunity to study embedding utilization and representation diversity (RQ4)



4. xLSTM for Recommenders:- Architectural design

The **architectural design** of mLSTM enhances standard LSTM through several mechanisms that make it more suitable for modern recommender systems:

- **Exponential Gating:** In mLSTM, the input and forget gates use exponential activation functions, instead of sigmoid functions. Normalization and Stabilization states.
- **Matrix Memory:** Instead of a vector cell state, mLSTM maintains a matrix valued memory. This enables the storage of structured information, meaning key value pairs stored via the outer products, which capture second order interactions between features.
- **Parallel Training:** The covariance update rule allows memory updates to be reformulated so that recurrence does not depend on memory mixing across time steps.
- **Covariance Update Rule:** Memory updates are based on outer products between projected value and key vectors, accumulating second order statistics in the memory matrix that encode interactions between the sequence elements and support associative retrieval.

Experiment Setup and Datasets

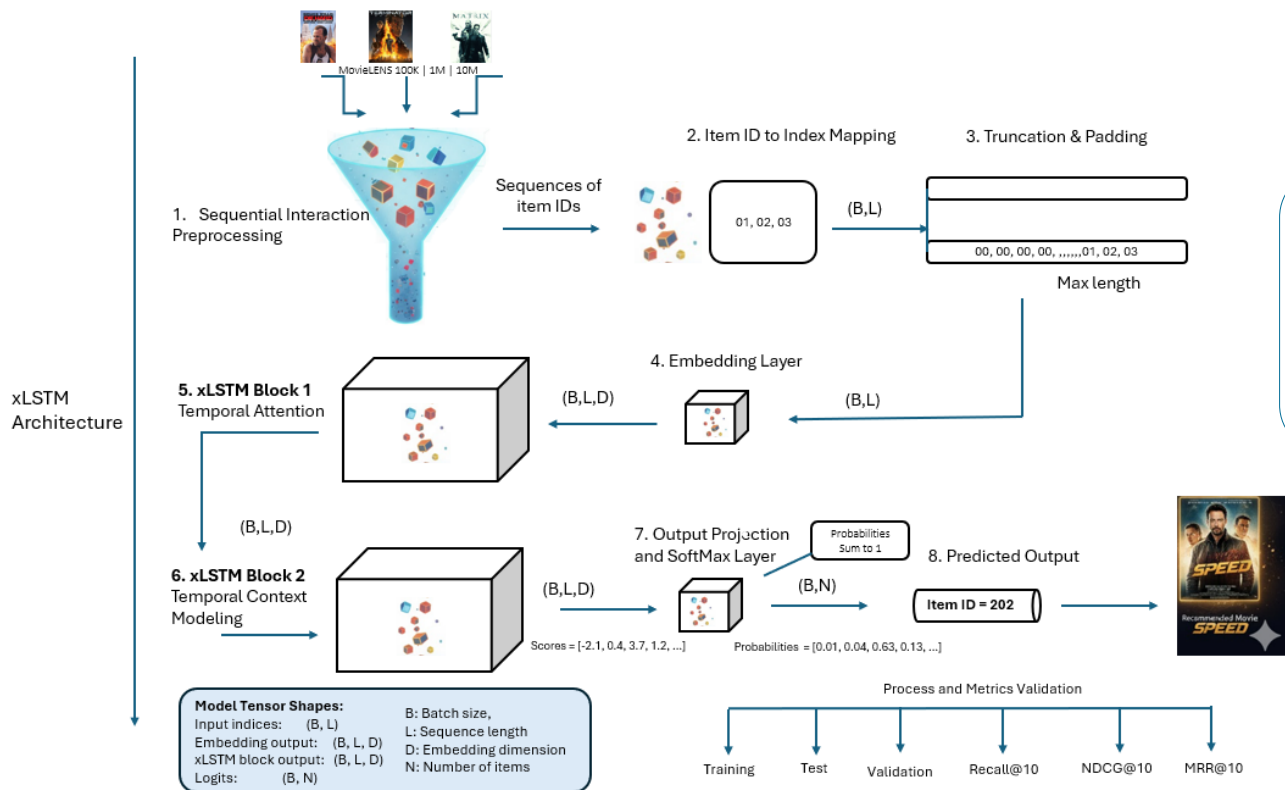
Five benchmark datasets widely used in the recommender systems:

- **MovieLens-100K, 1M, and 10M:** Publicly available under the Group Lens license, enabling benchmarking under dense user item. .
- **MUSIK4all:** Large-scale music interaction dataset (≈ 228 M events, 119k users) with play counts and timestamps. Released by JKU.
- **Amazon Product Data (Software subset):** Publicly available review and rating data capturing millions of user–item SW interactions.

Table 4.1: Experimental datasets.

Dataset	Users	Items	Interactions	Avg. len.	Sparsity
ML-100K	943	1,682	100,000	106.05	0.9369
ML-1M	6,040	3,416	999,611	165.49	0.9515
ML-10M	71,567	10,681	10,000,054	139.74	0.9869
Music For All	4584	51,291	2,533,266	552.6	0.9991
Amazon SW	3,75,147	21,663	4,59,436	-	0.9892

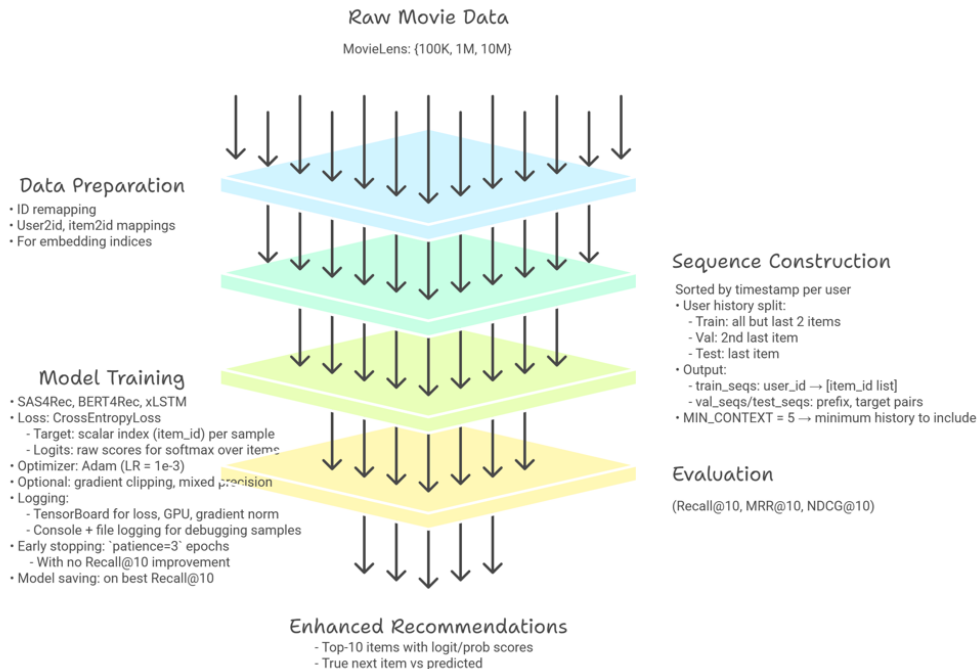
4. Proposed Architecture: xLSTM for Sequential Recommendation



Overall Architectural Setup:

- **Input 5 datasets:** MovieLens-100K, 1M, 10M, MUSIK4all, Amazon Software
- **3 sequence lengths:** 32, 64, 128
- **3 random seeds** per setup
- **Metrics:** Recall@10, MRR@10, NDCG@10 to measure ranking accuracy

Data Pipeline For Recommenders



Data Splitting Protocol and Experiment

- **Training:** All interactions except the final two ($t = 1$ to $N - 2$).
- **Validation Set:** Predict the $(N - 1)$ th item. [2]
- **Test Set:** Predict the final item using the full sequence.
- **Sequence Length:** 32, 64, 128
- **Total Experiments:** 84 ML Experiments, 3 seeds
- **Evaluation Metrics:** Hit Rate@10, MRR@10: NDCG@10

Experiment Model Results

Scaling Characteristics and Computational Trade-offs:

- On **MovieLens-100K**, xLSTM underperforms relative to BERT4Rec and SAS4Rec (e.g., HitRate@10 ~10.5%, compared to SAS4Rec ~12.8%).
- On **MovieLens-1M**, xLSTM's Recall@10 rises significantly (~26%), nearly converging with BERT4Rec (~27.7%), leveraging xLSTM's internal gating and memory mechanisms.
- xLSTM** incurs computational overhead (~1.5x–2x higher epoch time) but yields *comparable or superior Recall in large-scale*.
- xLSTM retains LSTM's linear O(n) temporal complexity**, augmented with partial recurrence and gating to improve long-range dependency.
- Architectural complexity scales with O(k)** parameters, where k governs memory depth, trading simplicity for enhanced sequence modeling capacity.

Table 5.1: Performance comparison across models, datasets, and sequence lengths (excluding training time, LR, and hit rate).

Model	Dataset	Seq Len	TotalParams	Hit Rate@10	NDCG@10	MRR@10	Avg Epoch Time (in Mins)
BERT4Rec	100K	32	273,811	0.1273 ± 0.0084	0.0620 ± 0.0044	0.0425 ± 0.0042	7.0
		64	275,859	0.1217 ± 0.0091	0.0582 ± 0.0056	0.0392 ± 0.0050	7.1
		128	279,955	0.1202 ± 0.0097	0.0584 ± 0.0042	0.0399 ± 0.0035	7.2
	1M	32	1,370,875	0.2766 ± 0.0149	0.1573 ± 0.0103	0.1209 ± 0.0088	95.4
		64	1,374,971	0.2752 ± 0.0155	0.1550 ± 0.0106	0.1184 ± 0.0090	95.3
		128	1,383,163	0.2777 ± 0.0174	0.1568 ± 0.0110	0.1200 ± 0.0092	101.3
	10M	32	8,712,886	0.3112 ± 0.0000	0.1494 ± 0.0000	0.1874 ± 0.0000	1,347.8
		64	8,721,078	0.2744 ± 0.0000	0.1232 ± 0.0000	0.1585 ± 0.0000	1,072.7
		128	8,737,462	0.3171 ± 0.0000	0.1513 ± 0.0000	0.1902 ± 0.0000	2,129.6
	Musik4All	32	13,600,220	0.3565 ± 0.0000	0.2626 ± 0.0000	0.2332 ± 0.0000	236.0
		64	13,604,316	0.3653 ± 0.0000	0.2668 ± 0.0000	0.2362 ± 0.0000	245.8
		128	13,612,508	0.3644 ± 0.0000	0.2720 ± 0.0000	0.2431 ± 0.0000	276.9
SAS4Rec	100K	32	5,985,824	0.1020 ± 0.0000	0.0732 ± 0.0000	0.0643 ± 0.0000	1.5
		64	5,989,920	0.1008 ± 0.0000	0.0712 ± 0.0000	0.0620 ± 0.0000	1.5
		128	5,998,112	0.0985 ± 0.0000	0.0692 ± 0.0000	0.0601 ± 0.0000	1.7
	1M	32	269,139	0.1278 ± 0.0098	0.0623 ± 0.0038	0.0427 ± 0.0028	5.9
		64	271,187	0.1247 ± 0.0106	0.0606 ± 0.0059	0.0414 ± 0.0051	6.0
		128	275,283	0.1283 ± 0.0106	0.0615 ± 0.0056	0.0415 ± 0.0044	6.0
	10M	32	1,353,339	0.2109 ± 0.0120	0.1191 ± 0.0083	0.0912 ± 0.0072	82.2
		64	1,357,435	0.1427 ± 0.0072	0.0798 ± 0.0053	0.0606 ± 0.0047	82.3
		128	1,365,627	0.0835 ± 0.0048	0.0461 ± 0.0032	0.0347 ± 0.0027	89.9
	Musik4All	32	8,645,046	0.2143 ± 0.0000	0.1246 ± 0.0000	0.0972 ± 0.0000	674.7
		64	8,653,238	0.1271 ± 0.0000	0.0713 ± 0.0000	0.0544 ± 0.0000	1,170.4
		128	8,669,622	0.0727 ± 0.0000	0.0400 ± 0.0000	0.0301 ± 0.0000	2,372.6
xLSTM	100K	32	13,582,427	0.3280 ± 0.0000	0.2397 ± 0.0000	0.2122 ± 0.0000	232.9
		64	13,586,523	0.3087 ± 0.0000	0.2285 ± 0.0000	0.2036 ± 0.0000	254.3
		128	13,594,715	0.2815 ± 0.0000	0.2107 ± 0.0000	0.1886 ± 0.0000	242.9
	Amazon Software	32	5,968,288	0.0050 ± 0.0000	0.0019 ± 0.0000	0.0009 ± 0.0000	1.3
		64	5,972,384	0.0050 ± 0.0000	0.0019 ± 0.0000	0.0009 ± 0.0000	1.3
		128	5,980,576	0.0050 ± 0.0000	0.0019 ± 0.0000	0.0009 ± 0.0000	1.7
	100K	32	269,188	0.1041 ± 0.0070	0.0509 ± 0.0036	0.0350 ± 0.0029	9.6
		64	269,188	0.1057 ± 0.0083	0.0522 ± 0.0050	0.0361 ± 0.0046	10.0
		128	269,188	0.1036 ± 0.0095	0.0511 ± 0.0044	0.0355 ± 0.0038	10.3
	1M	32	1,376,904	0.2625 ± 0.0146	0.1492 ± 0.0102	0.1148 ± 0.0089	143.1
		64	1,376,904	0.2603 ± 0.0151	0.1483 ± 0.0104	0.1143 ± 0.0089	147.0
		128	1,376,904	0.2605 ± 0.0155	0.1479 ± 0.0107	0.1136 ± 0.0092	164.2
	10M	32	8,698,176	0.3138 ± 0.0000	0.1910 ± 0.0000	0.1533 ± 0.0000	1,218.7
		64	8,698,176	0.3216 ± 0.0000	0.1962 ± 0.0000	0.1577 ± 0.0000	2,208.3
		128	8,698,176	0.3183 ± 0.0000	0.1937 ± 0.0000	0.1555 ± 0.0000	4,239.2
xLSTM	Musik4All	32	13,558,664	0.3496 ± 0.0000	0.2591 ± 0.0000	0.2309 ± 0.0000	701.5
		64	13,558,664	0.3439 ± 0.0000	0.2568 ± 0.0000	0.2296 ± 0.0000	1101.2
		128	13,558,664	0.3505 ± 0.0000	0.2605 ± 0.0000	0.2324 ± 0.0000	1851.4
	Amazon Software	32	5,973,896	0.0829 ± 0.0000	0.0600 ± 0.0000	0.0528 ± 0.0000	4.1
		64	5,973,896	0.0842 ± 0.0000	0.0598 ± 0.0000	0.0523 ± 0.0000	6.7
		128	5,973,896	0.0846 ± 0.0000	0.0619 ± 0.0000	0.0548 ± 0.0000	12.1

Experiment Model Results – Best Performing

Hit Rate@10 - Best Performing Configurations:-

Model Wise	Dataset	Seq Len	Hit Rate@10	NDCG@10	MRR@10
Bert4Rec	10M	128	0.317	0.15	0.19
Sas4Rec	10M	128	0.214	0.12	0.010
xLSTM	10M	128	0.318	0.19	0.15

NDCG@10 - Best Performing Configurations:-

Model Wise	Dataset	Seq Len	Hit Rate@10	NDCG@10	MRR@10
Bert4Rec	1M	32	0.276	0.157	0.12
Sas4Rec	10M	32	0.214	0.12	0.010
xLSTM	10M	64	0.318	0.19	0.15

Recal@10:- xLSTM: highest / tied top highest, BERT4Rec: highest / very close

NDCG@10:- xLSTM (achieved high NDCG@10) is better at producing **consistently good rankings across the Top-10 list**,

MRR@10:- BERT4Rec (achieved high MRR@10) is **better at placing the correct item at the very top**.

MRR@10 - Best Performing Configurations:-

Model Wise	Dataset	Seq Len	Hit Rate@10	NDCG@10	MRR@10
Bert4Rec	10M	128	0.317	0.15	0.19
Sas4Rec	10M	32	0.214	0.12	0.010
xLSTM	10M	64	0.318	0.19	0.15

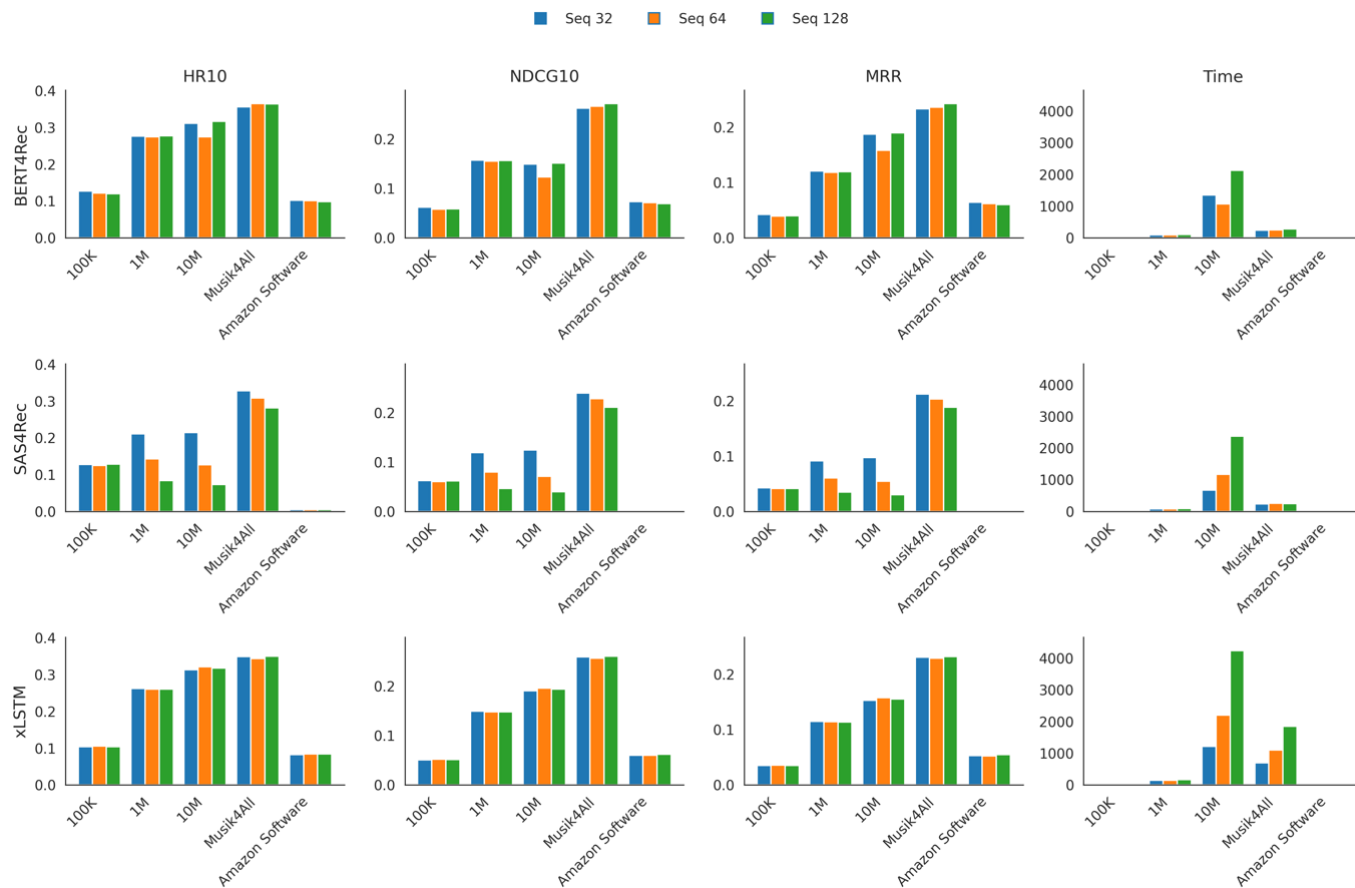
5. Findings With Respect to Research Questions

xLSTM evaluated under a novel configuration for Sequential recommenders; observed performance on various conditions.

- **RQ1 — Performance Scaling Across Dataset Sizes:** xLSTM demonstrates a clear positive scaling trend. While performance on the smallest dataset (ML 100K) drags the Transformer models, **xLSTM significantly improves as the interaction histories emerge**. On MovieLens 10M, xLSTM **reaches Recall@10 values around 31.8 percent**, converging closely with BERT4Rec, indicating that its gating mechanisms and the enhanced memory structures leverage medium scale datasets effectively.
- **RQ2 — Effects of Sequence Length and Embedding Size:** Experiments across sequence lengths of 32, 64, and 128 show that xLSTM increasing performance variance as sequences grow longer, reflecting its sensitivity to temporal window size. xLSTM **maintains stronger temporal fidelity in long sequences** due to its recurrent gating structure. Larger embedding dimensions strengthen this effect on large datasets, while offering limited benefit in sparse or in the short history domains.
- **RQ3 — Accuracy vs. Computational Efficiency Trade-offs:** xLSTM introduces a measured trade-off between accuracy and computational cost. Training times are typically 1.5×–2× longer than other baselines, and **inference speed is moderate**—faster than deep Transformer architectures yet slower than lightweight recurrent models. However, xLSTM **avoids the quadratic attention bottleneck of Transformers**.
- **RQ4 — Embedding Utilization, Saturation, and Representational Diversity:** Embedding geometry analyses reveal that **xLSTM makes substantially more effective use of embedding space** than Transformer baselines. While BERT4Rec and SAS4Rec exhibits the anisotropic embedding structures driven by the popularity bias, xLSTM produces nearly isotropic embeddings with **lower hubness, higher intrinsic dimensionality, and more uniform variance distribution**. CKA similarity studies further show that the xLSTM **learns fundamentally different**, sequence-oriented embedding structures rather than compressing items along global similarity axes.

Overall, xLSTM demonstrates strong scaling behavior (RQ1), clear sensitivity to sequence length and embedding size (RQ2), meaningful efficiency-accuracy trade-offs (RQ3), and superior embedding utilization compared to Transformer (RQ4).

Experiment Model Results



Popularity Bias and Fairness

These findings relate to RQ3 and provide additional insight into model behavior beyond accuracy metrics. **To assess whether a model favors frequently watched or highly rated items**, we analyze the distribution of its predicted movies.

If a small number of items dominate the top-10 recommendations across users, this indicates **popularity bias**.

Recall@10 — grouped bars by model and popularity bucket

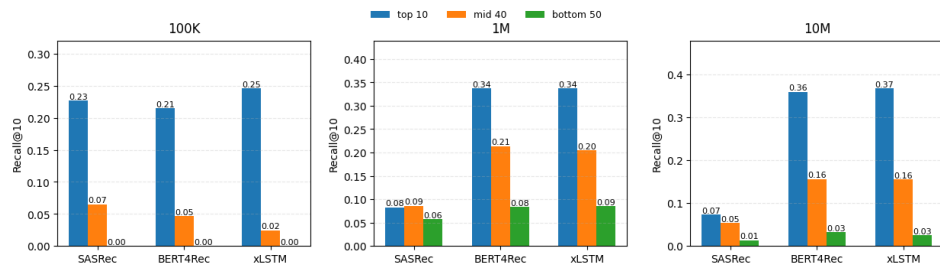
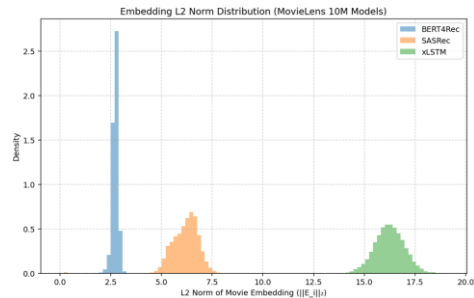


Table 5.3: Popularity-Bucketed Metrics Across Models and Datasets

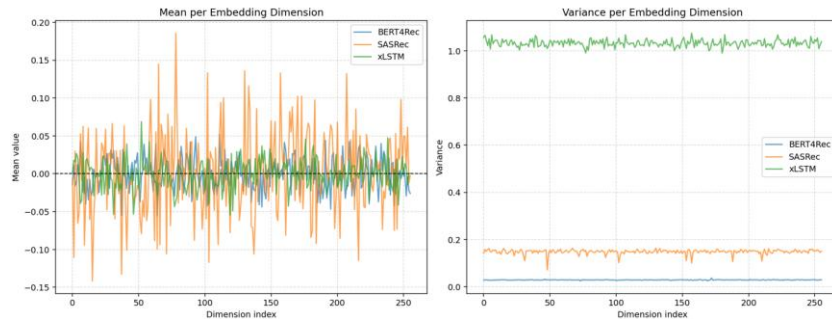
Model	Dataset	Seq Len	Bucket	Recall@10	MRR@10	NDCG@10
BERT4Rec	100K	128	top_10	0.2150	0.0714	0.1048
BERT4Rec	100K	128	mid_40	0.0471	0.0176	0.0243
BERT4Rec	100K	128	bottom_50	0.0000	0.0000	0.0000
BERT4Rec	1M	128	top_10	0.3373	0.1568	0.1991
BERT4Rec	1M	128	mid_40	0.2139	0.0797	0.1108
BERT4Rec	1M	128	bottom_50	0.0839	0.0376	0.0482
BERT4Rec	10M	128	top_10	0.3589	0.1728	0.2165
BERT4Rec	10M	128	mid_40	0.1562	0.0594	0.0818
BERT4Rec	10M	128	bottom_50	0.0321	0.0128	0.0172
SASRec	100K	128	top_10	0.2272	0.0689	0.1055
SASRec	100K	128	mid_40	0.0651	0.0178	0.0287
SASRec	100K	128	bottom_50	0.0000	0.0000	0.0000
SASRec	1M	128	top_10	0.0828	0.0354	0.0465
SASRec	1M	128	mid_40	0.0863	0.0303	0.0433
SASRec	1M	128	bottom_50	0.0579	0.0298	0.0362
SASRec	10M	128	top_10	0.0730	0.0293	0.0394
SASRec	10M	128	mid_40	0.0541	0.0211	0.0288
SASRec	10M	128	bottom_50	0.0138	0.0066	0.0083
xLSTM	100K	128	top_10	0.2460	0.0806	0.1185
xLSTM	100K	128	mid_40	0.0246	0.0072	0.0112
xLSTM	100K	128	bottom_50	0.0000	0.0000	0.0000
xLSTM	1M	128	top_10	0.3369	0.1536	0.1964
xLSTM	1M	128	mid_40	0.2049	0.0776	0.1071
xLSTM	1M	128	bottom_50	0.0855	0.0377	0.0488
xLSTM	10M	128	top_10	0.3670	0.1818	0.2253
xLSTM	10M	128	mid_40	0.1551	0.0600	0.0820
xLSTM	10M	128	bottom_50	0.0254	0.0107	0.0141

A-D: Embedding Geometry Analysis:-

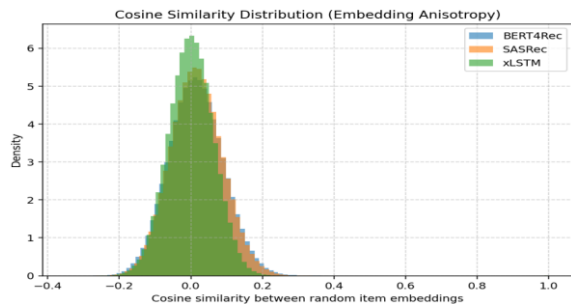


A. L2 Norm:- Supports RQ4 by showing whether larger embedding spaces are being effectively utilized or whether models fall into degenerate patterns such as overly sharp magnitude distributions.

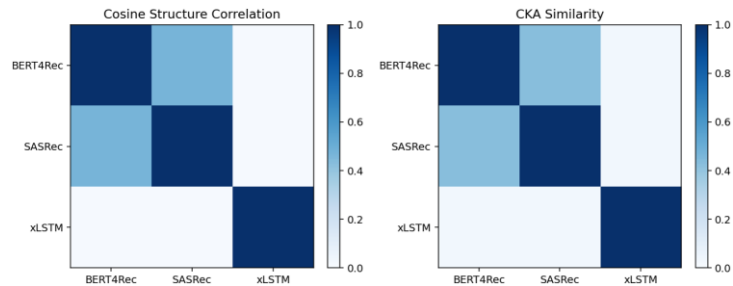
Embedding Mean and Variance per Dimension (MovieLens 10M Models)



B. Mean and variance reveal how evenly each embedding dimension is used, helping detect saturation, redundancy, or collapsed dimensions. RQ4 by showing whether xLSTM distributes information more effectively across the embedding space compared to baseline models.



C. Cosine-structure correlation and CKA similarity. BERT4Rec and SAS4Rec display high cross-model similarity (Corr = 0.471, CKA = 0.427), indicating that Transformers learn closely aligned geometric structures driven by semantic and co-occurrence patterns. xLSTM is almost orthogonal to both Transformer models (Corr $\approx$ 0.01, CKA $\approx$ 0.03). **xLSTM forms a unique, sequence-oriented embedding space.**



E. Embedding Geometry Analysis

Row 1 — Eigenvalue Spectrum (Decay Curves)

BERT4Rec, SAS4Rec:- Show steep eigenvalue decay, meaning most information lies in a few dominant directions.

xLSTM exhibits a slower decay, using more dimensions and more distributed. Higher intrinsic dimensionality and less collapse compared to Transformers.

Row 2 — Cumulative Explained Variance

BERT4Rec and SAS4Rec reach ~90% explained variance within ~50 components, showing strong redundancy. xLSTM requires ~200 components, demonstrating broader feature usage and more expressive embeddings. Its gradual slope confirms higher effective dimensionality and reduced rank collapse.

Row 3 — Hubness and Popularity Bias

Transformers: Highly skewed hubness, few items dominate nearest neighbour lists, reflecting popularity bias.

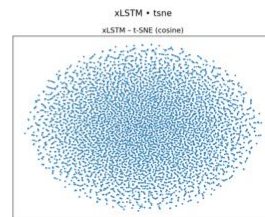
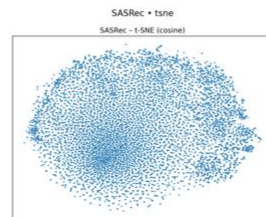
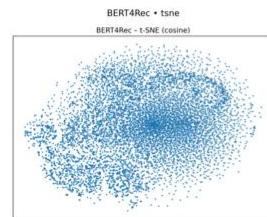
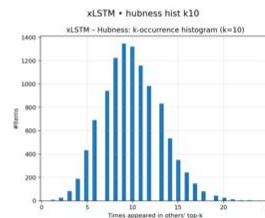
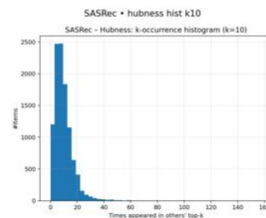
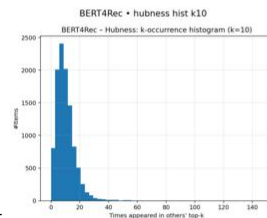
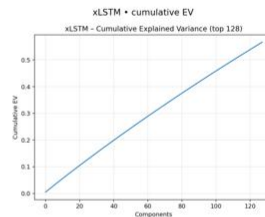
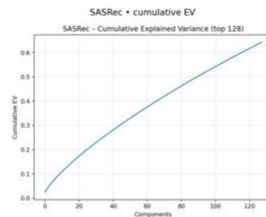
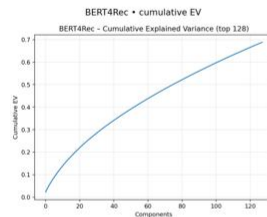
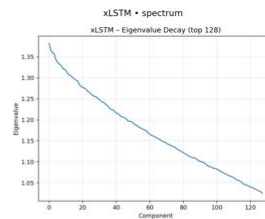
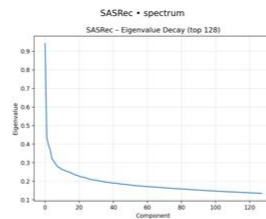
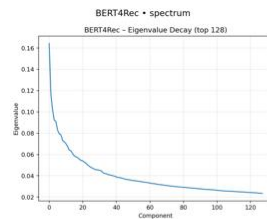
xLSTM: Flatter, more balanced distribution with much lower hubness (Gini ≈ 0.18). Better long-tail representation and fairer similarity structure.

Row 4 — t-SNE Embedding Visualization

BERT4Rec and SAS4Rec: Dense, blob-like clusters, showing anisotropy and popularity-driven collapse.

xLSTM: More diffuse, evenly distributed structure, indicating higher isotropy. Broader geometry supports more stable retrieval and better generalization

Models such as BERT4Rec apply Layer Norm extensively, leading to tightly controlled embedding norms. xLSTM applies normalization less.

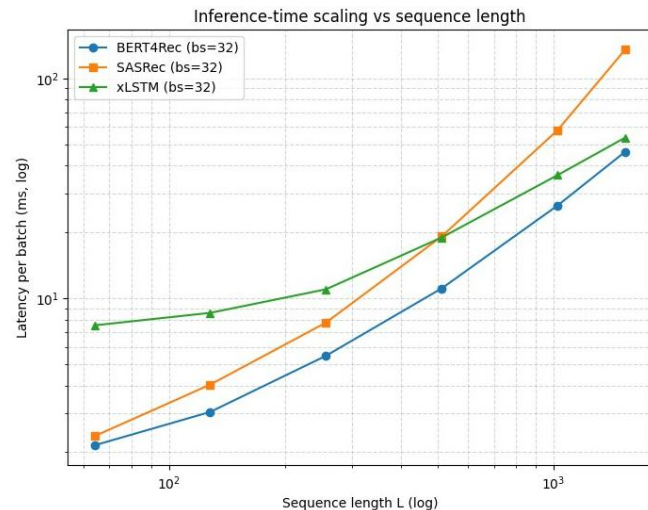


Model Parameters and Inference Speed

5.1.6 Key Observations on Model Parameters

Table 5.3: Layer-wise Trainable Parameter Breakdown for xLSTM Model Across MovieLens Datasets with Item Counts and Memory Estimates

Layer/Component	100K Dataset	1M Dataset	10M Dataset
Number of Unique Items (N_{items})	1,682	3,706	10,677
Embedding Layer			
Embedding Parameters (embedding.weight)	107,712	474,496	2,733,568
Backbone Block o			
norm_mlstm.weight	64	128	256
mlstm_layer.q.weight	2,048	8,192	32,768
mlstm_layer.k.weight	2,048	8,192	32,768
mlstm_layer.v.weight	4,096	16,384	65,536
ogate_preact.weight	4,096	16,384	65,536
igate_preact.weight	256	256	2,048
igate_preact.bias	2	2	8
fgate_preact.weight	256	256	2,048
fgate_preact.bias	2	2	8
multihead_norm.weight	128	128	256
out_proj.weight	4,096	16,384	65,536
norm_ffn.weight	128	128	256
ffn_proj_up_gate.weight	12,288	49,152	180,224
ffn_proj_up.weight	12,288	49,152	180,224
ffn_proj_down.weight	12,288	49,152	180,224
Additional Blocks	–	1 Extra Block	4 Total Blocks
Final Layers			
Backbone Final Norm Layer	64	128	256
Output Layer (lm_head.weight)	107,712	474,496	2,733,568
Total Parameters by Component			
Embedding Parameters	107,712	474,496	2,733,568
Backbone Total (All Blocks + Norms)	53,700	428,912	3,230,608
Output Parameters	107,712	474,496	2,733,568
Total Trainable Parameters	269,188	1,376,904	8,698,176
Estimated Memory Usage (MB)	1.03	5.25	33.2



Inference:- Sequence lengths from 64 to 1536 at a fixed batch size of 32. Latency, throughput, and log-log scaling exponents were computed to characterize how inference cost grows with sequence length.

xLSTM shows higher latency than Transformers for short sequences but much flatter growth as sequence length increases. Its effective scaling exponent ($\alpha \approx 0.64$) indicates sublinear runtime growth, reflecting chunk wise recurrence.

xLSTM narrows the latency gap at long sequences ($L \approx 1024-1536$), where attention-based models degrade more rapidly.

Limitations

The conclusions drawn from RQ1–RQ4 must be understood within the constraints of our experimental design, datasets, and modeling scope.

Dataset Scope: Our findings on MovieLens (100K–10M) and MUSIK4All, which are static and domain-specific. These datasets do not capture real world dynamics such as catalog drift, shifting session intent, or extreme scale, scaling trends may not fully generalize.

Sequence-Length Dependence: xLSTM performs best on long user histories (ML-1M/10M), whereas its advantages diminish on short-history datasets (ML-100K, Amazon Software). This limits applicability for platforms where user interactions are sparse or session-based.

Embedding Utilization: The geometric insights from RQ4 (e.g., isotropy, higher intrinsic dimensionality) are based on benchmark datasets with fixed vocabularies. Real-world systems with millions of frequently changing items may lead to different embedding behavior.

Production Gap: MovieLens and MUSIK4All do not reflect production feedback loops, popularity amplification, or adversarial interactions. Therefore, our interpretations of popularity bias and fairness may differ in live recommenders.

Despite these limitations, RQ1–RQ4) that xLSTM offers meaningful advantages in long-sequence modeling, embedding utilization, and representation diversity. These findings form a solid foundation for future work on larger, more dynamic datasets and indicate promising potential for xLSTM-based architectures in real-world recommender systems

7. Conclusion

- We benchmarked xLSTM against strong baselines (GRU4Rec, SAS4Rec, BERT4Rec) across multiple datasets (MovieLens-100K/1M/10M, MUSIK4All, Amazon Software). Evaluation was conducted using top-k ranking metrics (Recall@10, MRR@10, NDCG@10), supplemented by runtime analysis and qualitative inspection of recommendation outputs.
- Our findings show that **xLSTM scales well with longer interaction histories**. On MovieLens-10M, it achieves a **Recall@10 of approximately 0.318**—comparable to strong Transformer baselines—and remains competitive as data size and sequence depth increase. However, accuracy exhibits greater variance as sequence length grows (32 $\rightarrow$ 64 $\rightarrow$ 128), indicating sensitivity.
- While BERT4Rec performs consistently strongly across settings, **xLSTM offers favorable memory scaling and stable training dynamics at larger scales**, with the trade-off of higher computational cost. This cost is most noticeable on small datasets like MovieLens-100K, where lighter Transformer models can match or exceed xLSTM's accuracy with lower runtime.
- We analyzed **Model Parameters, Embedding Geometry, Inferencing** and addressed practical challenges—cold start and popularity bias. Popularity analyses and qualitative evaluations with insight beyond metrics and the contexts where models performs best.
- Overall, the xLSTM provides a strong foundation for scalable sequential recommendation: **effective at medium to large scale, memory efficient, and training stable**, while acknowledging weaker performance on the smallest datasets and offering efficiency suitable for practical deployment

Bibliography

- [1] Maximilian Beck, Korbinian Pöppel, Markus Spanring, Andreas Auer, Oleksandra Prudnikova, Michael Kopp, Günter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. *xLSTM: Extended Long Short-Term Memory*. 2024. arXiv: 2405.04517 [cs.LG].
- [2] D. Roy and M. Dutta. "A systematic review and research perspective on recommender systems". In: *Journal of Big Data* 9.59 (2022). DOI: 10.1186/s40537-022-00592-5.
- [3] Markus Schedl. "Personality and Perception of Affect in Music". In: *Proceedings of the 19th International Society for Music Information Retrieval Conference (ISMIR)*. Correlation of TIPI traits with perceived music emotions. 2018.
- [4] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. "BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer". In: *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM)* (2019), pp. 1441–1450. DOI: 10.1145/3357384.3357895.
- [5] Wang-Cheng Kang and Julian McAuley. "Self-Attentive Sequential Recommendation". In: *Proceedings of the 2018 IEEE International Conference on Data Mining (ICDM)* (2018), pp. 197–206. DOI: 10.1109/ICDM.2018.00035.
- [6] Markus Schedl, Peter Knees, Gerhard Widmer, et al. "MUSIK4all: A Large-Scale Music Dataset with User Interaction Histories". In: *Proceedings of the 23rd International Society for Music Information Retrieval Conference (ISMIR)*. 2022, pp. 887–894.
- [7] Deepak Kumar, Oleg Lesota, George Zerveas, Daniel Cohen, Carsten Eickhoff, Markus Schedl, and Navid Rekabsaz. "Parameter-efficient Modularised Bias Mitigation via Adapter Fusion". In: *arXiv preprint arXiv:2302.06321* (2023).
- [8] Lukas Hauzenberger, Shahed Masoudian, Deepak Kumar, Markus Schedl, and Navid Rekabsaz. "Modular and On-demand Bias Mitigation with Attribute-Removal Subnetworks". In: *arXiv preprint arXiv:2205.15171* (2022).
- [9] F. Maxwell Harper and Joseph A. Konstan. "The MovieLens Datasets: History and Context". In: *ACM Transactions on Interactive Intelligent Systems (TiiS)* 5.4 (2015), pp. 1–19. DOI: 10.1145/2827872.
- [10] Pablo Castells and Dietmar Jannach. *Recommender Systems: A Primer*. 2023. arXiv: 2302.02579 [cs.LG].
- [11] Sepp Hochreiter and Jürgen Schmidhuber. "Long Short-Term Memory". In: *Neural Computation* 9.8 (1997), pp. 1735–1780.
- [12] Dongfang Deng, Zixuan Lu, Qiang Liu, and Sepp Hochreiter. *xLSTM-Mixer: Multivariate Time Series Forecasting by Mixing via Scalar Memories*. 2024. arXiv: 2410.16928 [cs.LG].
- [13] Yifan Wu, Zixuan Lu, Dongfang Deng, and Sepp Hochreiter. *xLSTM Time: Long-term Time Series Forecasting With xLSTM*. 2024. arXiv: 2407.10240 [cs.LG].
- [14] Arpita Vats, Vinija Jain, Rahul Raja, and Aman Chadha. *Exploring the Impact of Large Language Models on Recommender Systems: An Extensive Review*. 2024. arXiv: 2402.18590 [cs.LR].
- [15] Jianmo Ni, Jiacheng Li, and Julian McAuley. "Justifying Recommendations using Distantly-Labeled Reviews and Fine-Grained Aspects". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2019, pp. 188–197. DOI: 10.18653/v1/D19-1018.
- [16] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan H. Kesha-van, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Q. Tran, Jonah Samost, Maciej Kula, Ed H. Chi, and Maheswaran Sathiamoorthy. *Recommender Systems with Generative Retrieval*. 2024.
- [17] A. Noorian, A. Harounabadi, and M. Hazratifard. "A sequential neural recommendation system exploiting BERT and LSTM on social media posts". In: *Complex Intelligent Systems* 10 (2024), pp. 721–744. DOI: 10.1007/s40747-023-01191-4.
- [18] H. Ahmadian Yazdi, S.J. Seyyed Mahdavi, and H. Ahmadian Yazdi. "Dynamic educational recommender system based on Improved LSTM neural network". In: *Scientific Reports* 14 (2024), p. 4381.
- [19] X. Yang and J. A. Esquivel. "Time-Aware LSTM Neural Networks for Dynamic Personalized Recommendation on Business Intelligence". In: *Tsinghua Science and Technology* 29.1 (2024), pp. 185–196. DOI: 10.26599/TST.2023.9010025.
- [20] Chhotelal Kumar and Mukesh Kumar. "Session-based recommendations with sequential context using attention-driven LSTM". In: *Computers and Electrical Engineering* 115 (2024), p. 109138. ISSN: 0045-7906. DOI: 10.1016/j.compeleceng.2024.109138.
- [21] Shivangi Gheewala, Shuxiang Xu, Soonja Yeom, and Sumbal Maqsood. "Exploiting deep transformer models in textual review based recommender systems". In: *Expert Systems with Applications* (2023). DOI: 10.1016/j.eswa.2023.121120.
- [22] RecBole. *RecBole: A Unified Framework for Recommendation Algorithms*. <https://recbole.io>. Accessed: 2025-07-14. 2020.
- [23] Edward Raff. "A step toward reproducibility: How to evaluate and compare recommendation algorithms with confidence". In: *Communications of the ACM* 64.4 (2021), pp. 72–80.
- [24] Matthew Hutson. "Artificial intelligence faces reproducibility crisis". In: *Science* 359.6377 (2018), pp. 725–726.
- [25] Xing Zhang, Kuan Ren, Tao Zhang, and Mu Li. "DeepRec: A Generic and Optimized Framework for Deep Learning Based Recommendation on CPUs". In: *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. 2021, pp. 3209–3219.

Other Slides

1. Topic Introduction

- Recommender systems aim to present users with the most relevant items by learning from large-scale interaction data. This work investigates the **extended Long Short-Term Memory (xLSTM) architecture** for sequential recommendation tasks.
- We **benchmark xLSTM against strong baselines** including Transformer-based models (BERT4Rec, SAS4Rec) and recurrent models, using five public datasets: MovieLens-100K, MovieLens-1M, MovieLens-10M, MUSIK4all, and Amazon Software. Evaluation uses top-k ranking metrics: Recall@10, MRR@10, NDCG@10, along with training time comparisons.
- xLSTM achieves competitive accuracy relative to Transformer baselines on medium and large-scale datasets. xLSTM offers favourable memory scaling and stable training dynamics. On smaller datasets, Transformer models slightly outperform xLSTM, showing capacity versus data size trade-offs.
- xLSTM is well-suited for large-scale recommender systems, balancing performance, memory efficiency, and training stability. Results show that xLSTM achieves competitive accuracy, especially on medium- and large-scale datasets, with favorable memory efficiency and stable trainings.
- Overall, xLSTM proves to be a strong, scalable sequence model suitable for modern large-scale recommender systems.

A. Recurrent Neural Networks (RNN)

Left-to-right manner, Unidirectional.

A vanilla RNN updates its hidden state by combining the current input with the previous state and then predicts the next item.

A simple RNN uses a recurrent structure with a single hidden state updated at each timestep. It **suffers from vanishing gradients**, which **limits its ability to learn long-term dependencies** in sequential recommendation tasks.

$$\mathbf{h}_t = \phi(W_x \mathbf{x}_t + W_h \mathbf{h}_{t-1} + \mathbf{b}_h), \quad (2.1)$$

$$\mathbf{s}_{t+1} = \mathbf{E}^\top \mathbf{h}_t, \quad (2.2)$$

$$\hat{\mathbf{y}}_{t+1} = \text{softmax}(\mathbf{s}_{t+1}), \quad (2.3)$$

$$\mathcal{L} = - \sum_t \log \hat{\mathbf{y}}_t[y_t] \quad (\text{cross-entropy over the true next item } y_t), \quad (2.4)$$

The limitations of RNNs directly justify exploring improved recurrent architectures such as xLSTM, and support addressing RQ3 by illustrating the accuracy–efficiency trade-offs present in early sequential models.

B. Long Short-Term Memory (LSTM)

LSTM is an advanced RNN variant with gates that control information flow, allowing it to capture long-range dependencies and mitigate vanishing gradient issues (the limitations previously noted for RNNs).

An LSTM contains **three gates** in its design:

- Forget Gate – decides what information to discard from the cell state.
- Input Gate – determines what new information to add to the cell state.
- Output Gate – controls what information is output from cell state as the hidden state.

The gates use:

- Sigmoid activation to regulate flow between 0 and 1, and
- Tanh activation updates the cell state and output.

$$\mathbf{i}_t = \sigma(W_i \mathbf{x}_t + \mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{b}_i), \quad (2.5)$$

$$\mathbf{f}_t = \sigma(W_f \mathbf{x}_t + \mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{b}_f), \quad (2.6)$$

$$\mathbf{o}_t = \sigma(W_o \mathbf{x}_t + \mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{b}_o), \quad (2.7)$$

$$\tilde{\mathbf{c}}_t = \tanh(W_c \mathbf{x}_t + \mathbf{U}_c \mathbf{h}_{t-1} + \mathbf{b}_c). \quad (2.8)$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \quad (2.9)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t). \quad (2.10)$$

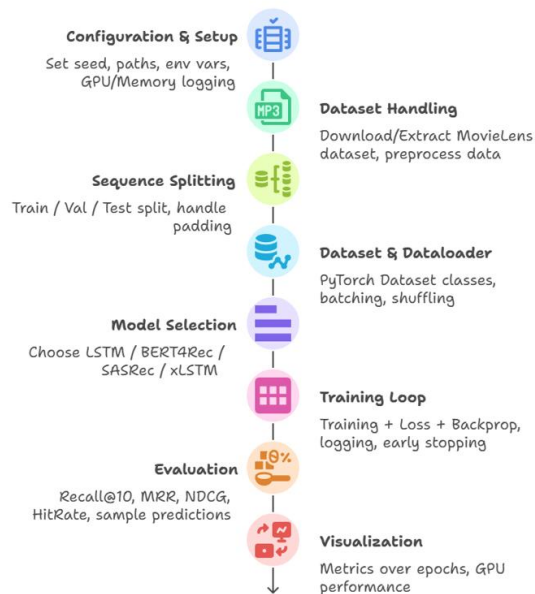
$$\mathbf{s}_{t+1} = \mathbf{E}^\top \mathbf{h}_t, \quad (2.11)$$

$$\hat{\mathbf{y}}_{t+1} = \text{softmax}(\mathbf{s}_{t+1}). \quad (2.12)$$

Overall, LSTMs add a gated memory cell to overcome vanishing gradients and maintain long-range context in user sequences.

Data Flow and Data Sampling Techniques

Streamlining Movie Recommendation Systems: A Step-by-Step Guide



Musik4All Datasets:

- A. Filter users <5 events,
- B. cap max 1000 events per user,
- C. hash-select ~5% of users,
- D. output to Parquet (~10M rows).

A. Random Sampling → Select a fraction p of rows uniformly (e.g., USING SAMPLE 1% in DuckDB); fast but may fragment user timelines.

B. Stratified Sampling → Sample proportionally within groups (e.g., per user/item category); implemented with group by + sample() in Pandas/Polars.

C. Systematic Sampling → Pick every k -th record after a random offset; efficient for ordered files but risky if patterns exist.

D. Time-based Sampling → Filter interactions by a timestamp window (e.g., WHERE ts >= NOW() - INTERVAL '90 days'); preserves temporal recency.

E. User-based Hash Sampling → Deterministic subset of users via hash (e.g., hash(user_id) % 100 = 0); keeps complete histories of selected users.

F. Per-user Last-K Sampling → Take last K events per user using window functions (ROW_NUMBER() OVER (PARTITION BY user ORDER BY ts DESC)); reduces data but preserves recency. We used this method for Musik4All datasets.

G. Storage formats → For scale, write sampled subsets to Parquet/ZSTD (columnar, compressed) for fast reloads vs. raw TSV/CSV.

H. Best practice → Use hash or last-K sampling for recommender research, time-based for evaluation splits, and combine with Parquet for speed.

Transfer Learning in Latent Factor Model

This section relates to RQ4 (embedding saturation), as transferability reflects how meaningful and generalizable the learned embeddings are. Using the MovieLens 1M and 10M datasets, transfer learning is performed by training models on one dataset (e.g., ML-1M) and then adapting or fine-tuning them on the other (e.g., ML-10M). **The goal is to transfer learned embeddings** and interaction patterns from the smaller (or larger) dataset to improve performance on the other, **leveraging shared user–item structures** across datasets.

In earlier analyses, we examined latent features from the best-performing models to understand embedding dimensions and similarities. In this transfer learning step, we **reuse these learned weights** and evaluate their performance on related datasets. Since **item IDs remain consistent** across MovieLens datasets, this enables direct comparison of embedding dimensions and transferred representations.

Transfer is carried out by reusing pretrained latent vectors or model weights and fine-tuning them on the target dataset while accounting for domain differences such as catalog size. Our results show that models pretrained on MovieLens 1M provide beneficial initialization when trained on MovieLens 10M, and vice versa, improving both accuracy and convergence speed

Bias and Fairness

Societal Bias: Although the model uses only movie ID sequences (as shown in Figure 3.2), the underlying MovieLens interactions reflect demographic viewing patterns such as age- or gender-based genre preferences. These latent biases enter indirectly through user behavior. Our results show that Transformer models amplify these tendencies through anisotropic embeddings and hub formation, causing certain genres or film types to dominate similarity neighborhoods. In contrast, xLSTM’s more isotropic embedding geometry reduces such effects.

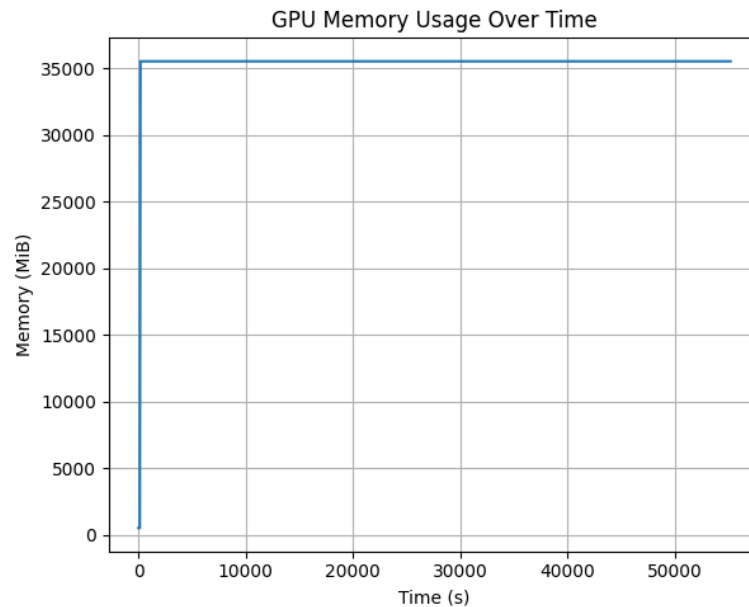
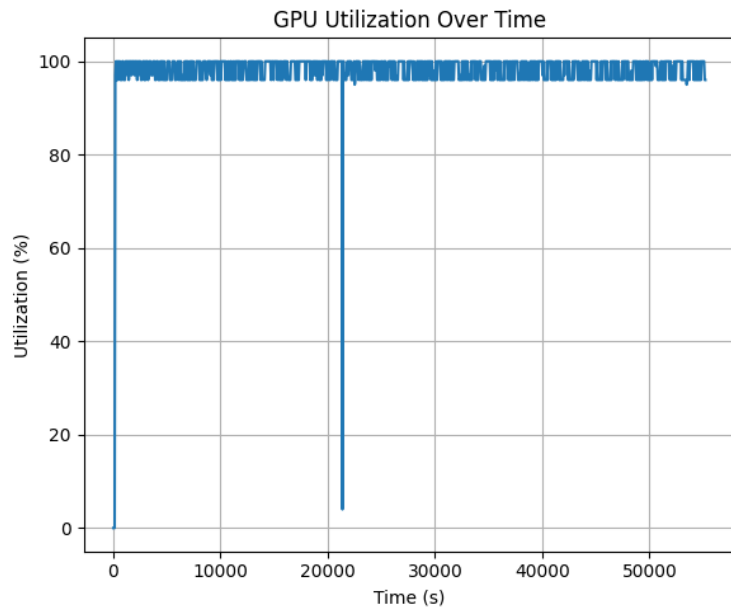
Statistical Bias: The MovieLens dataset exhibits strong popularity bias (Figure 5.10), where blockbuster movies receive disproportionately more interactions. This bias propagates into model behavior: BERT4Rec and SAS4Rec compress their embeddings around popular items, as indicated by the spectrum and hubness analyses. This mirrors exposure and feedback-loop biases documented in recommender systems.

Individual Fairness: Ideally, users with similar movie-watching patterns should receive similar recommendations. However, in MovieLens, Transformer models overweight popular films and may treat similar users inconsistently depending on the amount of long-tail content they consume. xLSTM’s more isotropic embedding space supports balanced neighbor retrieval, resulting in more consistent recommendations for similar users.

Group Fairness: Even though the model trains only on item ID sequences, demographic differences in MovieLens still influence interaction patterns. Popularity-heavy models tend to favor user groups aligned with blockbuster genres. Transformer embeddings reinforce this bias by over-emphasizing popular items. xLSTM’s more isotropic geometry reduces such dominance, leading to more balanced item exposure across different user groups.

Popularity Calibration: Because MovieLens is heavily skewed toward popular items, models must avoid over-recommending already dominant titles. Our results show that xLSTM’s reduced hubness and higher intrinsic dimensionality yield a more calibrated popularity distribution, aligning more closely with each user’s true preference mix than Transformer-based baselines

GPU Performance: A100 in Colab



Architectural Setup:

- **Input 5 datasets:** MovieLens-100K, 1M, 10M, MUSIK4all, Amazon Software
- **3 sequence lengths:** 32, 64, 128
- **3 random seeds** per setup
- **Total experimental:** 81 experimental runs
- **Training time:** 200-250 Hours in A100 GPU
- **Logging :** Tensor board

2. Existing Model Architectures For Sequential Recommenders

- Recurrent Neural Networks (RNN)
- Long Short-Term Memory (LSTM)
- Gated Recurrent Unit for Recommendation (GRU4Rec)
- Self-Attentive Sequential Recommendation (SAS4Rec)
- Bidirectional Encoder Representations for Recommendation (BERT4Rec)

Table 4.2: Model architecture comparison across key dimensions.

Feature	LSTM	xLSTM	BERT4Rec	SASRec
Attention	No	Yes	Yes	Yes
Bi-directional	No	Yes	Yes	Yes
Training Stability	Medium	High	Medium	High
Inference Speed	Fast	Moderate	Slow	Moderate

Feature	RNN	LSTM	SASRec	BERT4Rec	xLSTM
Attention Mechanism	No	No	Yes (Self-Attention)	Yes (Full Transformer Attention)	Yes (Sparse Attention)
Bidirectional Processing	No	No	Yes (conceptually, via attention patterns)	Yes	Yes
Memory Design	Single hidden state	Scalar memory cell with gates	Transformer hidden states	Transformer hidden states	Hybrid scalar + matrix memory (sLSTM + mLSTM)
Long-Range Dependency Modeling	Weak (vanishing gradients)	Moderate	Strong	Very Strong	Strong
Training Stability	Low-Medium	Medium	High	Medium	High
Inference Speed	Fastest	Fast	Moderate-Fast	Slow	Moderate
Parallelizability	Very Low	Low	High	High	Moderate
Sequence Direction	Left-to-right	Left-to-right	Left-to-right self-attention	Bidirectional (masked modeling)	Bidirectional
Best For	Simple short-sequence baselines	Short-mid sequences; stable baseline	Efficient transformer baseline for sequential rec.	Highest accuracy and semantics	Balanced accuracy + efficiency; scalable models
Limitations	Struggles with long range; unstable	Slow on long seq.; limited parallelism	Less semantic depth than BERT4Rec	Heavy/slow inference	More complex; newer architecture

Evaluations Metrics, Loss Function and Hyperparameters

Recall@10: Measures the proportion of times the correct next item appears in the top-10 predicted list. It captures model coverage and hit rate.

$$\text{Recall@K} = \frac{|(\text{relevant items}) \cap (\text{top-K predicted items})|}{|(\text{relevant items})|} \quad (4.1)$$

NDCG@10 (Normalized Discounted Cumulative Gain): Considers both the relevance and position of items in the top-10 list, rewarding higher placements of correct predictions.

$$\text{NDCG@K} = \frac{1}{|U|} \sum_{u=1}^{|U|} \frac{1}{\text{IDCG}_u@K} \sum_{i=1}^K \frac{\mathbb{I}[\text{item}_i \text{ is relevant}]}{\log_2(i+1)} \quad (4.3)$$

MRR@10 (Mean Reciprocal Rank): Evaluates the average of reciprocal ranks of the first relevant item. It reflects how high the correct item is ranked in the list.

$$\text{MRR@K} = \frac{1}{|U|} \sum_{u=1}^{|U|} \frac{1}{\text{rank}_u} \quad (4.2)$$

- **Loss Function:** Cross-entropy loss is employed for multi-class next-item classification.

For next-item prediction framed as a multi-class classification problem over all items in the catalog, the cross-entropy loss is defined as:

$$\mathcal{L}_{CE} = - \sum_{i=1}^N \log \left(\frac{e^{z_{i,y_i}}}{\sum_{j=1}^C e^{z_{i,j}}} \right) \quad (4.4)$$

where:

N is the number of training examples (users or sequences),

C is the total number of candidate items (classes),

$z_{i,j}$ is the logit score (pre-softmax output) for class j for example i ,

y_i is the true class (next item index) for example i .

Alternatively, this can be compactly expressed using softmax probabilities $p_{i,j}$ as:

$$\mathcal{L}_{CE} = - \sum_{i=1}^N \log(p_{i,y_i}) \quad \text{where} \quad p_{i,j} = \frac{e^{z_{i,j}}}{\sum_{k=1}^C e^{z_{i,k}}} \quad (4.5)$$

Table 4.2: xLSTM and DataLoader hyperparameters per dataset.

Dataset	Embed Dim	Heads	Blocks	Batch Size	Workers
100K	64	2	1	128	2
1M	128	2	2	128	2
10M	256	8	4	1024	16
Musik4All	128	2	2	1024	2
Amazon Software	128	2	2	1024	2